

**SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE**  
**Fakulta elektrotechniky a informatiky**

**POIT**  
**Záverečné zadanie ku skúške**  
**Výstražný systém**

**Šk. rok 2025/2026**  
**Bc. Viktor Barczy**

# 1. Úvod a cieľ projektu

Cieľom projektu je vytvoriť IoT výstražný alarmový systém na monitorovanie vzdialenosti pomocou reálneho senzora. Systém deteguje priblíženie objektu pod nastavenú prahovú vzdialenosť. Ak je táto podmienka splnená, aktivuje sa alarmová signalizácia pomocou RGB LED diódy a bzučiaka.

Projekt využíva dosku ESP32-C3, ultrazvukový senzor HY-SRF05, RGB LED diódu a bzučiak. ESP32-C3 zabezpečuje meranie vzdialenosti a ovládanie akčných členov. Serverová časť je vytvorená v jazyku Python pomocou frameworku Flask a beží v Linuxovom prostredí vo VirtualBoxe, ktoré funkčne nahrádza Raspberry Pi. Komunikácia medzi serverom a ESP32-C3 prebieha cez USB Serial rozhranie.

Používateľ môže cez webové rozhranie nastaviť interval merania, prahovú vzdialenosť pre alarm a farby LED diódy pre normálny aj alarmový stav. Webová aplikácia zobrazuje aktuálnu hodnotu, graf, ručičkový ukazovateľ, zoznam alarmových meraní a archív z databázy. Údaje sa ukladajú do SQLite databázy aj do CSV súboru.

GITHUB repozitár: [https://github.com/ViktorBarczy/Poit\\_Zadanie/tree/main](https://github.com/ViktorBarczy/Poit_Zadanie/tree/main)

## 2. Použité hardvérové a softvérové prostriedky

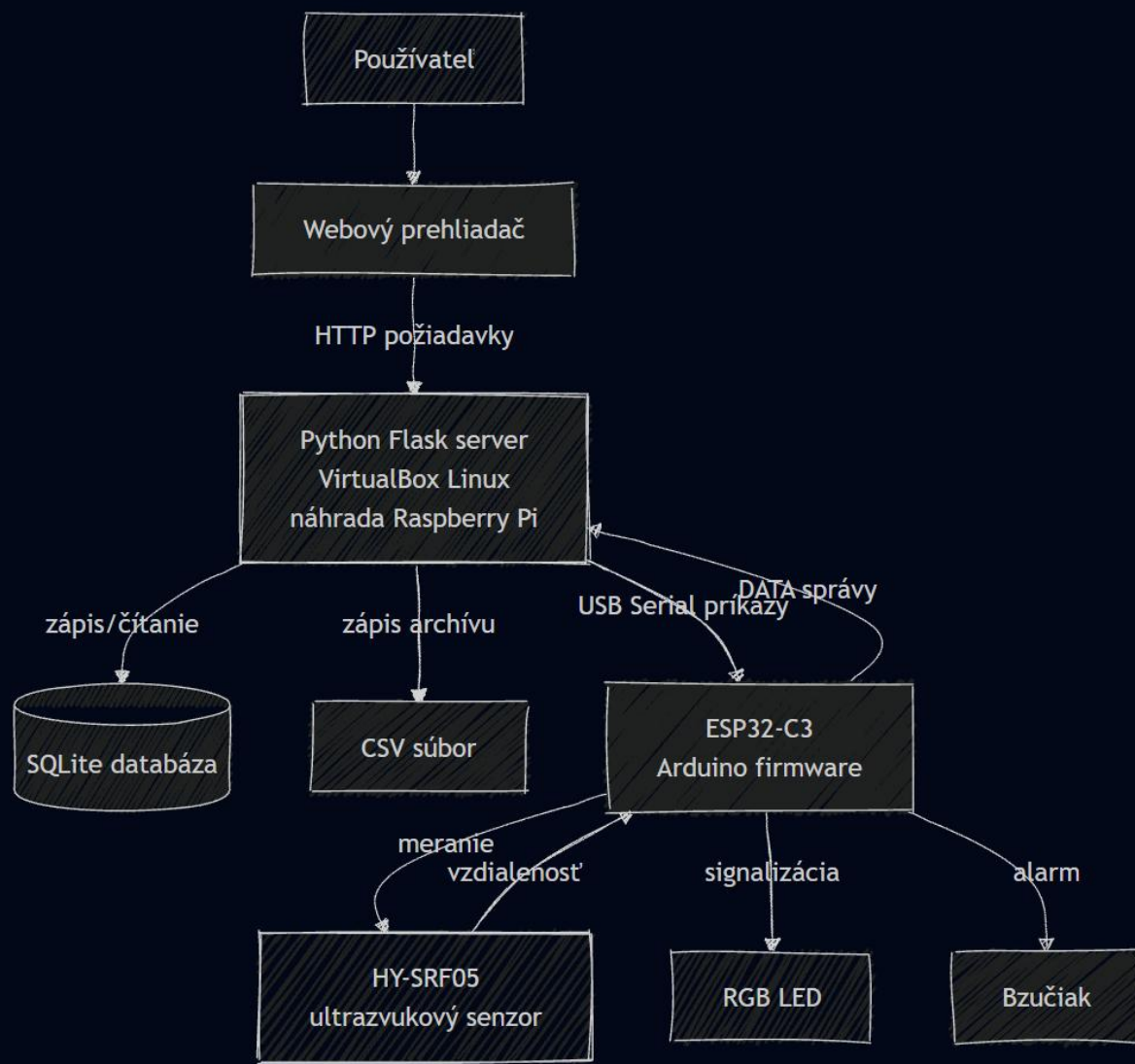
| Kategória                  | Použité prvky                                 |
|----------------------------|-----------------------------------------------|
| Mikrokontrolér             | ESP32-C3                                      |
| Senzor                     | Ultrazvukový senzor HY-SRF05                  |
| Akčné členy                | RGB LED dióda, bzučiak                        |
| Server                     | Linux vo VirtualBoxe ako náhrada Raspberry Pi |
| Programovací jazyk servera | Python 3                                      |
| Webový framework           | Flask                                         |
| Klient                     | HTML, CSS, JavaScript, Chart.js               |
| Databáza                   | SQLite                                        |
| Súborová archivácia        | CSV súbor                                     |
| Komunikácia                | USB Serial, 115200 baud                       |
| Verziónovanie              | GitHub                                        |

### 2.1 Zapojenie hardvéru

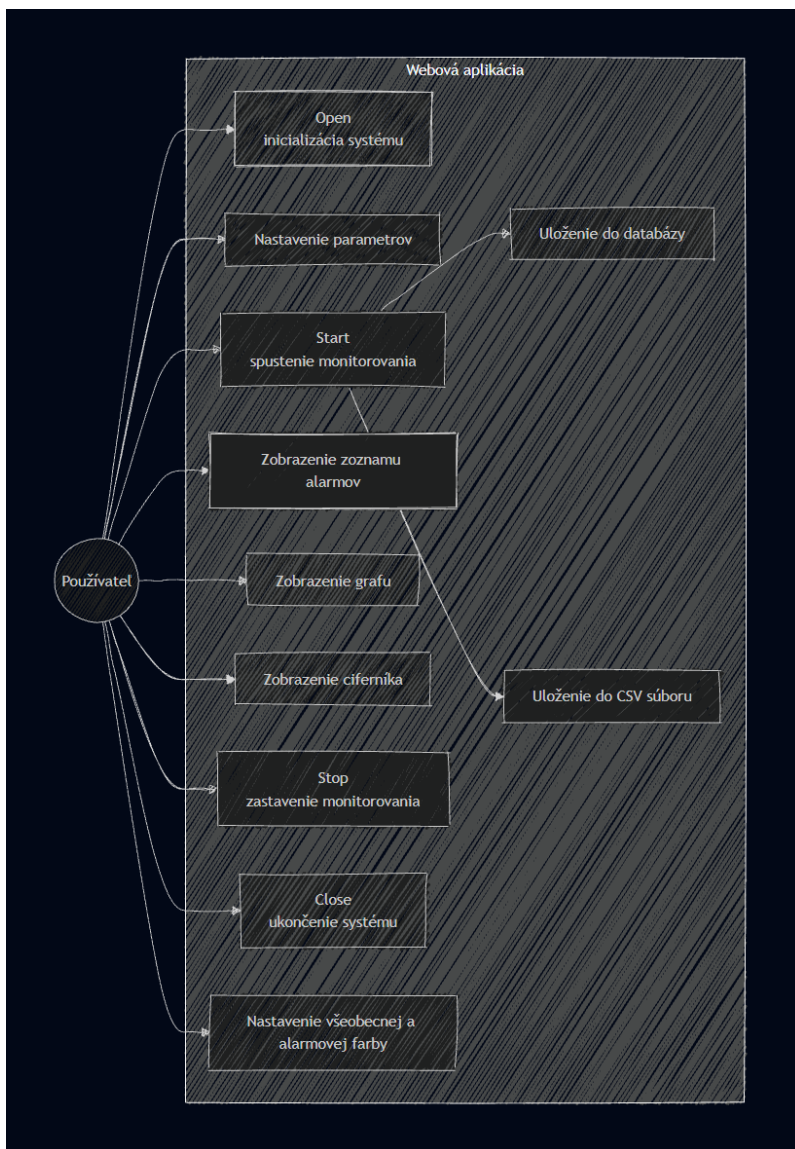
| Prvok         | Pin ESP32-C3 | Popis                                 |
|---------------|--------------|---------------------------------------|
| HY-SRF05 TRIG | GPIO 7       | Výstupný impulz pre spustenie merania |
| HY-SRF05 ECHO | GPIO 8       | Vstup dĺžky spätného impulzu          |
| RGB LED RED   | GPIO 10      | Červená zložka RGB LED                |
| RGB LED GREEN | GPIO 20      | Zelená zložka RGB LED                 |
| RGB LED BLUE  | GPIO 21      | Modrá zložka RGB LED                  |
| Bzučiak       | GPIO 3       | Zvuková signalizácia alarmu           |

## 3. Návrh architektúry aplikácie

Aplikácia je rozdelená na hardvérovú, serverovú a klientskú časť. Hardvérová časť zabezpečuje meranie vzdialenosti a ovládanie LED diódy s bzučiacom. Serverová časť prijíma dáta zo sériovej linky, poskytuje API pre webové rozhranie a archivuje údaje. Klientská časť poskytuje ovládacie prvky a vizualizáciu údajov.



### 3.1 UML - diagram komponentov



## 4. Komunikačný protokol medzi serverom a ESP32

Komunikácia prebieha cez USB Serial linku s rýchlosťou 115200 baud. Server odosiela jednoznakové príkazy, prípadne príkazy s hodnotou. ESP32 odpovedá stavovými hláseniami alebo meranými údajmi.

| Príkaz   | Význam                                                    |
|----------|-----------------------------------------------------------|
| O        | Open - inicializácia hardvéru a prepnutie do stavu OPENED |
| S        | Start - spustenie monitorovania                           |
| T        | Stop - zastavenie monitorovania                           |
| C        | Close - deaktivácia hardvéru a ukončenie spojenia         |
| P        | Status - vyžiadanie aktuálneho stavu                      |
| I1000    | Nastavenie intervalu merania na 1000 ms                   |
| D30      | Nastavenie prahovej vzdialenosti na 30 cm                 |
| G1 až G4 | Nastavenie všeobecnej farby LED                           |
| A1 až A4 | Nastavenie alarmovej farby LED                            |

| Odpoveď ESP32 | Príklad                                      | Popis                                    |
|---------------|----------------------------------------------|------------------------------------------|
| ESP32_READY   | ESP32_READY                                  | ESP32 je pripravené po štarte            |
| STATE         | STATE:MONITORING                             | Informácia o aktuálnom stave             |
| DATA          | DATA:24.15,30.00,1,MONITORING                | Meranie vzdialenosti, prah, alarm a stav |
| STATUS        | STATUS:MONITORING,1000,30.00,24.15,GREEN,RED | Kompletný stav zariadenia                |
| LOG           | LOG:Monitoring started                       | Informačná správa                        |
| ERROR         | ERROR:START_NOT_ALLOWED_PRESS_OPEN_FIRST     | Chybová správa                           |

## 5. Serverová časť aplikácie

Serverová časť je implementovaná v súbore raspberry/app.py. Používa framework Flask, knižnicu pyserial na komunikáciu cez sériový port a sqlite3 na prácu s databázou. Po stlačení tlačidla Open sa otvorí sériový port, spustí sa čítacie vlákno a do ESP32 sa odošle príkaz O. Pri monitorovaní čítacie vlákno prijíma riadky začínajúce DATA:, spracuje ich a uloží do databázy aj CSV súboru.

| Endpoint             | Metóda | Funkcia                                              |
|----------------------|--------|------------------------------------------------------|
| /                    | GET    | Zobrazenie hlavnej webovej stránky                   |
| /api/open            | POST   | Otvorenie sériového spojenia a inicializácia systému |
| /api/settings        | POST   | Nastavenie intervalu a prahovej vzdialenosti         |
| /api/settings/colors | POST   | Nastavenie všeobecnej a alarmovej farby RGB LED      |
| /api/start           | POST   | Spustenie monitorovania                              |
| /api/stop            | POST   | Zastavenie monitorovania                             |
| /api/close           | POST   | Ukončenie spojenia a deaktivácia systému             |
| /api/state           | GET    | Získanie aktuálneho stavu aplikácie                  |
| /api/recent          | GET    | Získanie posledných meraní pre graf a tabuľku        |
| /api/archive/db      | GET    | Výpis archívu z databázy                             |
| /api/archive/events  | GET    | Výpis udalostí systému                               |

### 5.1 Nastavenie portu vo VirtualBoxe

Virtuálny Linux musí vidieť dosku ESP32 ako sériové zariadenie. Vo VirtualBoxe sa zapne USB Controller a pridá sa USB filter pre zariadenie ESP32, CP210x, CH340 alebo Espressif USB JTAG/Serial. V Linuxe sa port overí príkazom:

```
ls /dev/ttyACM* /dev/ttyUSB*
```

Typický port je /dev/ttyACM0 alebo /dev/ttyUSB0. V aplikácii je predvolený port /dev/ttyACM0, prípadne ho možno zmeniť vo webovom formulári alebo premennou prostredia SERIAL\_PORT.

## 6. Klientska časť aplikácie

Klientska časť je vytvorená v súbore raspberry/templates/index.html. Obsahuje ovládacie tlačidlá, formuláre pre parametre, aktuálnu numerickú hodnotu, graf meranej vzdialenosti, ciferník a tabuľku meraní. Graf je realizovaný pomocou knižnice Chart.js a ciferník je vykresľovaný pomocou elementu canvas.

| Ovládací prvok | Funkcia                                                       |
|----------------|---------------------------------------------------------------|
| Open           | Inicializuje systém, otvorí sériové spojenie a aktivuje ESP32 |

|                   |                                                |
|-------------------|------------------------------------------------|
| Uložiť parametre  | Nastaví interval merania a prahovú vzdialenosť |
| Start             | Odštartuje monitorovanie/reguláciu             |
| Stop              | Zastaví monitorovanie/reguláciu                |
| Close             | Deaktivuje systém a zatvorí spojenie           |
| Všeobecná farba   | Nastaví farbu LED pri normálnom stave          |
| Alarm farba       | Nastaví farbu LED pri alarme                   |
| Načítať DB archív | Zobrazí uložené merania z databázy             |

Zoznam meraní môže byť upravený tak, aby zobrazoval iba záznamy, pri ktorých bol alarm zapnutý. Graf a ciferník pritom môžu zobrazovať všetky aktuálne merania.

## 7. Databáza a súborová archivácia

Aplikácia archivuje merané údaje dvoma spôsobmi: do SQLite databázy a do CSV súboru. Databáza sa nachádza v priečinku `raspberry/data/iot_archive.sqlite3` a CSV súbor v `raspberry/data/iot_archive.csv`. Dáta sa ukladajú automaticky pri prijatí každého merania zo zariadenia ESP32.

| Tabuľka measurements | Význam                              |
|----------------------|-------------------------------------|
| id                   | Primárny kľúč záznamu               |
| created_at           | Čas uloženia merania na serveri     |
| esp_millis           | Čas z ESP32, ak je dostupný         |
| distance_cm          | Nameraná vzdialenosť v centimetroch |
| threshold_cm         | Nastavená prahová hodnota           |
| alert                | Informácia, či bol alarm aktívny    |
| led                  | Použitá farba LED                   |
| buzzer               | Stav bzučiaka                       |
| interval_ms          | Interval merania                    |

| Tabuľka events | Význam                                       |
|----------------|----------------------------------------------|
| id             | Primárny kľúč udalosti                       |
| created_at     | Čas udalosti                                 |
| event_type     | Typ udalosti, napr. OPEN, START, STOP, CLOSE |
| message        | Textová správa udalosti                      |
| interval_ms    | Interval v čase udalosti                     |
| threshold_cm   | Prahová hodnota v čase udalosti              |

## 8. Program pre ESP32-C3

Program pre ESP32-C3 je uložený v priečinku `esp32_sensor_node`. Doska ESP32-C3 pracuje ako hardvérový uzol alarmového systému. Zabezpečuje meranie vzdialenosti pomocou senzora HY-SRF05, vyhodnotenie alarmovej podmienky a ovládanie RGB LED diódy a bzučiaka. So serverovou časťou komunikuje cez USB Serial rozhranie. Zo servera prijíma príkazy na otvorenie systému, spustenie a zastavenie monitorovania, nastavenie prahovej vzdialenosti, nastavenie periódy merania a nastavenie farieb LED diódy pre normálny a alarmový stav.

Meranie vzdialenosti sa vykonáva pomocou pinov TRIG a ECHO. Na pin TRIG sa odošle 10 mikrosekundový impulz a na pine ECHO sa meria dĺžka odozvy. Z tejto hodnoty sa vypočíta vzdialenosť v centimetroch. Samotné meranie sa nevykonáva nepretržite, ale v nastaviteľnej perióde, ktorú používateľ zadáva vo webovej aplikácii. Táto hodnota sa odošle zo servera do ESP32-C3 cez USB Serial príkaz.

Ak je vzdialenosť menšia ako nastavený prah, aktivuje sa alarmová farba LED a bzučiak. V normálnom stave svieti všeobecná farba LED a bzučiak je vypnutý. Program používa enum stavy CLOSED, OPENED, MONITORING a STOPPED, vďaka čomu je riadenie systému prehľadnejšie.

## 9. Používateľská príručka

### 9.1 Spustenie aplikácie

Najprv sa vo VirtualBoxe nastaví USB passthrough pre dosku ESP32. Následne sa v Linuxe spustí aplikácia v priečinku raspberry. Príkazy pre spustenie sú:

```
cd raspberry
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python app.py
```

Po spustení servera sa v prehliadači otvorí adresa:

<http://127.0.0.1:5000>

### 9.2 Bežný pracovný postup používateľa

1. Pripojiť ESP32-C3 k počítaču cez USB a priradiť ho do VirtualBoxu.
2. Spustiť Linux vo VirtualBoxe a overiť port príkazom `ls /dev/ttyACM* /dev/ttyUSB*`.
3. Spustiť Python Flask aplikáciu.
4. Vo webovej aplikácii zadať správny sériový port a baudrate 115200.
5. Kliknúť na tlačidlo Open.
6. Nastaviť interval merania a prahovú vzdialenosť.
7. Nastaviť všeobecnú a alarmovú farbu LED.
8. Kliknúť na Start a sledovať graf, ciferník a alarmové merania.
9. Kliknutím na Stop zastaviť meranie.
10. Kliknutím na Close ukončiť spojenie a deaktivovať systém.

## 10. Testovanie a vyhodnotenie bodov zadania

| Bod zadania              | Realizácia v projekte                                                               |
|--------------------------|-------------------------------------------------------------------------------------|
| 1. Open                  | Tlačidlo Open otvorí sériový port, spustí čítacie vlákno a pošle príkaz O do ESP32. |
| 2. Nastavenie parametrov | Používateľ nastavuje interval merania, prahovú vzdialenosť a farby LED.             |
| 3. Start                 | Tlačidlo Start odošle parametre a príkaz S na spustenie monitorovania.              |
| 4. Výpis údajov          | Merania sa zobrazujú v tabuľke, pričom možno zobrazovať iba alarmové záznamy.       |
| 5. Grafy                 | Vzdialenosť a prahová hodnota sa zobrazujú v čiarovom grafe.                        |
| 6. Ciferníky             | Aktuálna vzdialenosť sa zobrazuje pomocou ručičkového ukazovateľa.                  |
| 7. Databáza              | Merania a udalosti sa ukladajú do SQLite databázy.                                  |
| 8. Súbor                 | Merania sa paralelne ukladajú do CSV súboru.                                        |
| 9. Stop                  | Tlačidlo Stop pošle príkaz T a zastaví monitorovanie.                               |
| 10. Close                | Tlačidlo Close pošle príkaz C a uzavrie sériové spojenie.                           |

## 12. Záver

Výsledkom projektu je funkčný IoT výstražný alarmový systém, ktorý spája reálny hardvér ESP32-C3 s webovou aplikáciou bežiacou v Linuxovom prostredí vo VirtualBoxe. Systém monitoruje vzdialenosť pomocou ultrazvukového senzora HY-SRF05 a pri poklese vzdialenosti pod nastavenú prahovú hodnotu aktivuje alarmovú signalizáciu pomocou RGB LED diódy a bzučiaka.

Webová aplikácia umožňuje inicializáciu systému, nastavenie parametrov, spustenie a zastavenie monitorovania, zmenu farieb pre normálny a alarmový stav, vizualizáciu údajov pomocou tabuľky, grafu a ručičkového ukazovateľa a archiváciu údajov do SQLite databázy aj CSV súboru.

Riešenie spĺňa základné princípy IoT architektúry, pretože obsahuje reálny senzor, akčné členy, serverovú časť, klientské webové rozhranie a dátové úložisko. Projekt zároveň ukazuje jednoduchý spôsob, ako je možné vytvoriť praktický výstražný systém ovládaný a monitorovaný cez webové rozhranie.